

C++基础：函数与运算符的重载

函数概念

- 从历史上说，函数的思想来源于子程序（如 **Basic** 语言中的子程序），把程序中反复出现的相同或相近的部分改写成子程序，可以大大缩短程序的长度。函数实际上是参数化的子程序。
- 从结构化程序设计（**SP**）的观点来看，函数绝不仅仅是为了缩短程序长度，更重要的是通过函数设计，可以把整个程序要完成的整体的复杂的计算任务，分解为一个个较小的、相对简单的子任务。这种模块化的程序易设计，易阅读，易调试，易维护，较少出错。
- 从运算的角度来讲，函数就是 **C++** 语言提供的由用户定义的运算。运算符是系统提供的运算，而函数是由用户自己定义的运算。
- 作为面向对象程序设计（**OOP**）语言的 **C++**，以类为核心，类由数据和方法组成，方法就是对数据的运算和处理，亦即类的函数成员。故函数设计同样是 **OOP** 的重要组成部分。

三次方程求根程序的设计

按照 Cardan 公式，计算三次方程 $x^3+px+q=0$ 的一个实根的公式为：

$$x_r = \sqrt[3]{-\frac{q}{2} + \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}}$$

- 两种设计思路：
 - 直接求解：计算 `croot` 的立方根的运算进行了两次，所以该计算程序要重复两次，当程序较长或计算次数更多时会更加复杂；更重要的是，当重复代码多了，逻辑结构会变得复杂而且难以捉摸。
 - 利用函数进行计算分解，然后再求解：们采用自顶向下逐步求精的方法，把一个复杂的问题分解为若干比较简单的子问题，可以减少编程的难度。使用函数进行分层分模块计算，可以大大缩短程序的长度，并且使程序的逻辑变得非常清晰。特别是当程序比较复杂时，可以使得程序显得更清晰，更容易维护。

三次方程求根程序的设计

为了从系数 p 、 q 计算实根 xr ，把以上公式的计算分解为下面几步：

(1) 令实数 $xr=A+B$ 。

(2) 令实数 A 、 B 分别为实数 R 、 S 的立方根。

(3) 令 $R=-q/2+a$ 、 $S=-q/2-a$ 。

(4) 令 $a=\text{sqrt}((q/2)*(q/2)+(q/3)*(q/3)*(q/3))$ 。

实际的计算过程为：

- 用第 (4) 步的公式计算得到 a 。
- 用第 (3) 步的公式计算得到 R 和 S 。
- 求出 R 和 S 的立方根 A 和 B 。
- 最后得到实根 xr 。

三次方程求根程序的设计

```
26 //Compute the result of x where x^3+px+q=0
27 √ float cardan(float p,float q){
28     float xr,root;
29     const float eps=1e-6;
30     // Compute sqrt of a=q/2* q/2+p/3* p/3* p/3
31     float a=sqrt(q/2* q/2+p/3* p/3* p/3);
32     // Compute cuberoot of -q/2+a
33     float croot = -q/2+a,x=croot;
34 √ do {
35         root=croot;
36         croot=(2* root+x/(root* root))/3;
37     } while(fabs(croot-root)>eps);
38     xr=croot;
39     // Compute cuberoot of -q/2-a
40     croot= -q/2-a; x=croot;
41 √ do {
42         root=croot;
43         croot=(2* root+x/(root* root))/3;
44     } while (fabs(croot-root)>eps);
45     xr+=croot;
46     cout<<endl<<"Cardan: The real root of the equation is "<<xr;
47     return xr;
48 }
```

直接求解

$$xr = \sqrt[3]{-\frac{q}{2} + \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}}$$

三次方程求根程序的设计

```
19 //Compute the result of x where x^3+px+q=0
20 ✓ float cardan_v2(float p,float q){
21     float xr, a=a=sqrt(q/2* q/2+p/3* p/3* p/3);
22     xr=cuberoot(-q/2+a)+cuberoot(-q/2-a);
23     cout<<endl<<"CardanV2: The real root of the equation is "<<xr;
24     return xr;
25 }
```



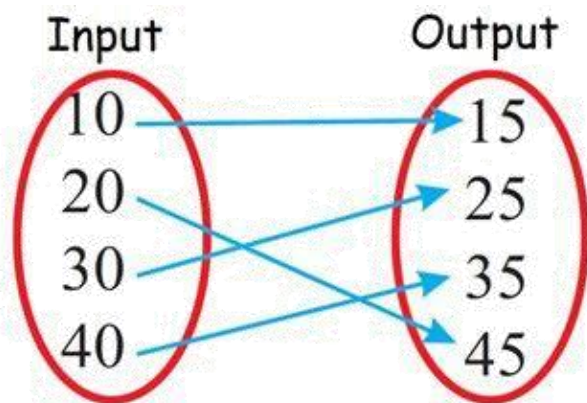
```
6 //compute cuberoot of x
7 float cuberoot (float x)
8 ✓ {
9     float root, croot;
10    const float eps=1e-6;
11    croot=x;
12    ✓ do{
13        root=croot;
14        croot=(2* root+x/(root* root))/3;
15    }while(fabs(croot-root)>eps);
16    return croot;
17 }
```

函数分层求解

$$xr = \sqrt[3]{-\frac{q}{2} + \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}} + \sqrt[3]{-\frac{q}{2} - \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}}$$

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



函数的说明

- C++程序允许两种函数说明语句的形式，分别称为**函数原型（或函数声明）**和**函数定义**。
- 函数原型（亦称函数声明）用来指出函数的名称、类型和参数，其格式为：

[<属性说明>]<类型><函数名>(<参数表>);

- 例如：

```
int add (int a, int b);
```

```
inline void swap (float & s, float & t);
```

```
void print (char *);
```


函数的说明

- C++程序允许两种函数说明语句的形式，分别称为**函数原型（或函数声明）**和**函数定义**。

- 函数原型（亦称函数声明）用来指出函数的名称、类型和参数，其格式为：

[<属性说明>]<类型><函数名>(<参数表>);

- **属性说明**：可缺省，一般可以是下面的关键字之一：inline、static、virtual、friend 等：
 - inline：表示该函数为内联函数。
 - static：表示该函数为静态函数。
 - virtual：表示该函数为虚函数。
 - friend：表示该函数为某类（class）的友元函数。

函数的说明

- C++程序允许两种函数说明语句的形式，分别称为**函数原型（或函数声明）**和**函数定义**。
- 函数原型（亦称函数声明）用来指出函数的名称、类型和参数，其格式为：

[<属性说明>]<类型><函数名>(<参数表>);

- **类型**：指函数的返回类型。C++语言规定除了特别情形（类的构造函数、析构函数）之外，所有函数都必须在说明中指出返回类型。一个函数可能有多个结果，不一定都用返回值的方式输出（还可通过全局变量、引用参数和指针等方式传出）计算结果，但函数调用作为一个表达式，表达式的值则只能是函数的返回值。

函数的说明

- C++程序允许两种函数说明语句的形式，分别称为**函数原型（或函数声明）**和**函数定义**。
- 函数原型（亦称函数声明）用来指出函数的名称、类型和参数，其格式为：

[<属性说明>]<类型><函数名>(<参数表>);

- 函数名：一个标识符。
- **参数表**：可能为空、为 `void` 或<类型><参数名>，<类型><参数名>…的形式。

例如：

```
main(); print(void); cuberoot(float x); add(int a, int b);
```

其中，函数原型中的参数表可忽略参数名，如：

```
int add(int, int); void swap(float &, float &);
```

函数的说明

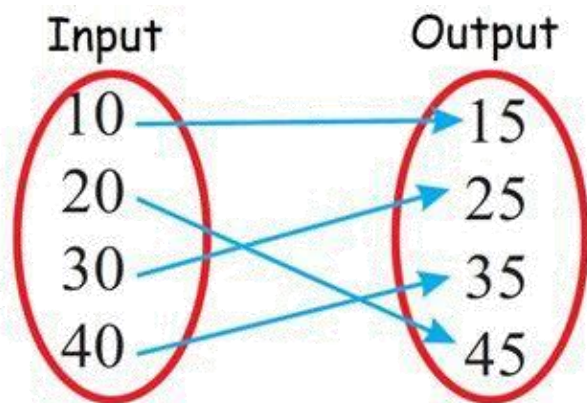
- 函数原型一般在两种情形下被使用：
 - 在程序中某函数的调用语句出现在该函数的定义之前，必须在调用语句之前列出函数原型。（注意C++程序所有的函数和变量定义声明都是从上往下的，如果在调用函数之前没有定义该函数名，即使是在其下一行才定义该函数名，那么都是不合法的）
 - 为了定义类简明清晰，一般把实现代码较长、逻辑较复杂的成员函数定义移到类说明之外，这时应把该函数的原型列于类说明之中。

函数的说明

- 函数定义与函数原型的主要区别是它还包括**函数体**，其格式为：
 [<属性说明>]<类型><函数名>(<参数表>)<函数体>
- 属性说明、类型、函数名与函数原型一致
- **参数表中不可省略参数名。**
- 函数体：由“{”和“}”括起来的复合语句，即程序块。
- 函数定义本身是一种说明语句，函数体则是一个复合语句。而在函数体内部又可能包含说明语句、表达式语句、控制语句和复合语句。

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
 - 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
 - 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



函数的调用

- 函数调用是对已定义函数的一次实际运行，与某类型的一个变量和某类（后文介绍）的一个对象类似，函数调用可以看作是函数定义的一个“实例”。
- 函数说明中的参数称为形式参数（形参），函数调用中的参数称为实际参数（实参），函数调用一般出现在表达式中或独立形成一个函数调用语句。
- 函数调用的两要素是函数名和实参表：

<函数名> (<实参表>)

实参表中的参数的类型、个数、顺序必须与函数说明的参数表（形参表）相一致。赋值型的实参可以是一个表达式。例如：

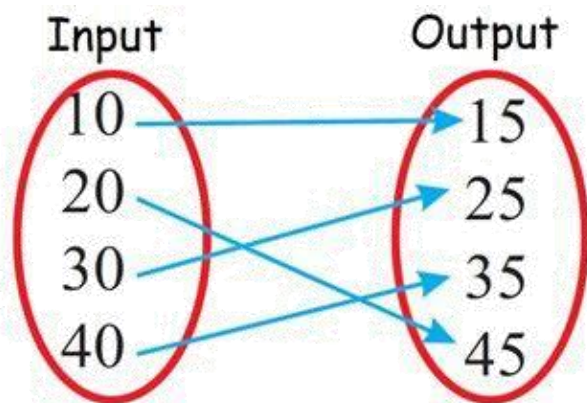
```
xr=cuberoot (-q/2+a)+cuberoot(-q/2-a);  
swap (a, b);
```

函数的调用

- 函数 `cuberoot(x)`调用，其具体的调用实施过程如下：
 - 根据调用语句中的函数名（`cuberoot`）在整个程序中搜索**同名函数定义**。（可能有同名函数重载的）
 - 对实参的个数、类型、顺序进行核对，判定是否与函数定义中的**形参表对应一致**，在上例中只有一个浮点型参数。（从同名函数中区分）
 - 根据参数的类型（值参数或引用参数）进行值参数的值传递或引用参数的“换名”，在上例中即是要把**实参表达式的值计算出来赋给形参 `x`**。
 - 运行函数体代码。
 - 返回调用点，并返回所要求的函数值，即返回计算结果 `croot` 的值。

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



函数的返回

- 函数的返回完成两项任务：
 - 把运行控制从函数体返回到函数调用点。在上例中就是在计算 $\text{cuberoot}(-q/2+a)$ 之后再返回到语句 $xr = \text{cuberoot}(-q/2+a) + \text{cuberoot}(-q/2-a)$; 的计算过程中。
 - 根据返回值要求，返回所需要的数据值。

函数的返回

- 函数的返回类型：
- 返回 `void` 类型。如果函数无返回值，应说明为 `void` 类型。例如：

```
void print( ) { cout<<"Hello World ! "; } void show( ) { cout<<"Wonderful C++!"; }
```

函数不需返回任何数据，一般仅形成一个函数调用语句。
- 返回数值类型。最常见的函数是返回一个数值的函数。例如：

```
int add (int a, int b) ; float cuberoot (float x) ;
```

这类函数的调用表达式可以出现在表达式语句中。当函数要返回的数值不止一个时，一般它可以以结构或类的形式，也可以以结构、数组或对象指针类型方式实现。

- 返回引用类型。值返回方式是 C 和 Pascal 语言中唯一的返回方式，C++ 语言提供的引用返回概念是其特有的一种很强的功能，**当定义函数把该函数说明为某类型的引用类型时，该函数调用后返回的不单是值，而是包含返回值的变量（或对象）。**

函数的返回

Code

```
51  ✓ int& add(int& a, int b){  
52      |     return a+=b;  
53      | }  
  
10      | int a=1,b=2;  
11      | add(a,b)=5;  
12      | cout<<"a="<<a<<" , b="<<b<<<endl;
```

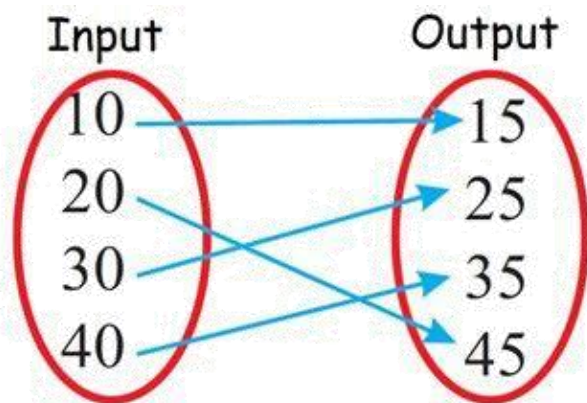
Run

问题：

- 1、能编译通过吗？
- 2、输出结果是什么？

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



函数的参数

- 函数的参数的设置和使用是函数设计中非常重要的部分。C++语言允许函数无参数，有一个或多个参数，而且还支持有不定个数参数的函数：
 - 无参函数：其函数说明为下列形式：`void print (void); int getx ( );`
 - 一个或多个参数：多数函数有一个或多个确定个数、确定类型和顺序的参数。定义时应注意参数表的组成。因为在C++语言中，不同的函数是根据函数名和参数表二者来区分的，只有二者完全一致才是同一函数。
 - 不定个数参数。有些应用问题中被处理的数据个数是变化的：
 - 使用数组或结构体，例如`void sort (int n, float * a);`这个函数可对n长的浮点数组进行排序，n是变化的；由于a是数组的首元指针，因此这个函数实际上可以对任意多个浮点数排序进行处理，起到了不定个数参数的效果。
 - 使用格式`void abc (int n,...);`来指定相同类型的n个参数，n是首地址。
 - C++语言还允许参数表中包含无名参数，主要是为了区分函数。例如：

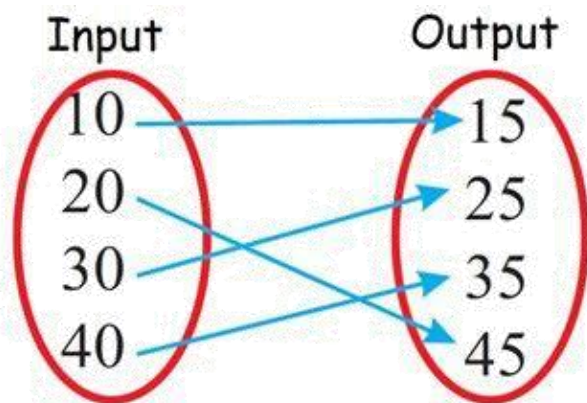
```
int f (int a, int b) {return  a+b*b;}
```

```
int f (int a, int b, int) {return  a*a+b;}
```

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归

- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



值调用与引用调用

- C++语言在进行函数调用时，对参数的处理有两种方式：赋值型和引用型，即值调用方式和引用调用方式。
- 赋值型：是普通的形式，在C语言中只有这种方式，所有的参数都是局部变量赋值，不会改变外部的值；
- 引用型：函数定义的参数表中，名字前加上符号&的参数为引用形参。引用型会改变传入参数的值。

值调用与引用调用

- 赋值调用方式在执行函数调用时，检查函数名及参数表之后，首先为值参数分配内存，然后计算各对应的实参表达式，并把计算的值赋给刚刚创建的参数变量，进而开始函数体的运行。
- 在函数定义的参数中，除了被声明为引用（&）的参数之外，其余所有类型的形参都属于赋值形参。
- 凡是赋值形参，在每次调用函数时，都必须为每一个赋值形参创建一个新的参数变量。
- 为赋值形参创建的参数变量是局限于函数体运行的局部变量，它作为该形参的一个实例，参加函数体程序块的本次运行，一旦运行完毕，这个参数变量就被撤销。

```
int n=5;  
cout<<multiple3(2)<<endl;  
cout<<multiple3(n)<<endl;  
cout<<multiple3(3 * n)<<endl;  
cout<<n<<endl;
```

```
int multiple3(int n){  
    n=n*3;  
    return n;  
}
```

值调用与引用调用

- 引用调用方式的引用形参：函数定义的参数表中，名字前加上符号&的参数为引用形参。
- 引用形参在调用过程中的参数传递机制不同于赋值形参。其要点如下：
 - 函数的调用语句中对应于引用形参的实参**必须是同一类型的变量，非变量的表达式则不允许。**
 - 参数传递的内容不是实参的值，而是地址，其实际的效果是令对应的引用形参在调用过程中，作为一个变量名指向作为实参的这个变量，与赋值形参的不同在这里体现出来，**在引用调用过程中并不创建新的参数变量。**
 - 在函数体程序块的运行中，引用形参每次出现后，由于它已经指向实参变量，因此相当于全用实参变量所代替。即起到了所谓的“换名”的作用。
 - 在函数体程序运行结束，控制转回调用点时，该引用形参与实参变量的对应关系也就终止了。**但是在调用过程中对于这个实参变量的所有处理和操作的结果，却保留了下来，这一点也是区别于赋值调用的。**

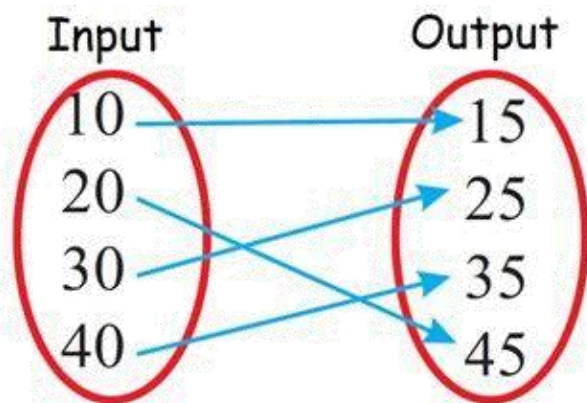
值调用与引用调用

- 设计函数在下面两种情形时，采取引用调用方式：
 - 需要改变某些变量的值（上述函数 **swap** 就是一例）。
 - 对于占内存较多的数据参数，为了不另建新的参数变量以节省内存，建议采用引用参数。为了保证实参不在函数中被修改，可在形参声明中加上 **const** 说明

```
void swap (int& a, int& b) {      int x=5, y=3, z=7;
    int temp=a;                  swap (x, y);           //结果 x=3, y=5
    a=b;                         swap (y, z);           //结果 x=3, y=7, z=5
    b=temp;
}
```

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



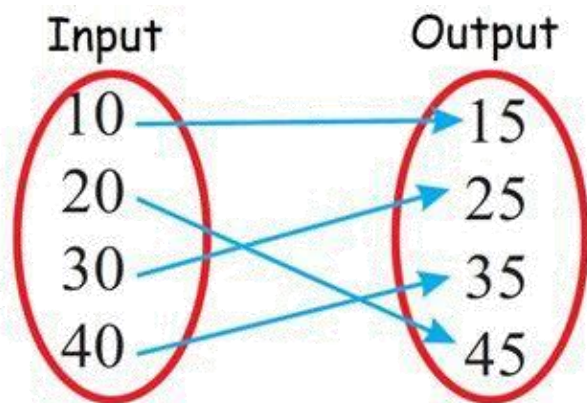
内联函数

- 在编译过程中，凡内联函数，系统把它的执行代码插入到该函数的每个调用点，从而使程序执行过程中，每次调用该函数时不需转移控制点，但该函数代码可能有多个拷贝出现在目标程序中。也就是说，把多次调用的函数说明为内联函数，会使目标程序占用的空间加大，而运行时间得到节省。
- 为了优化程序、提高程序的运行效率，一般把函数体短小而又频繁调用的函数声明为内联函数。
- 利用编译预处理的宏定义方式，也可以实现类似于内联函数的功能。不过，内联函数的方式更为方便、可靠。一个发生在编译前，一个发生在编译后。

<code>inline int MAX (int a, int b) {if (a>b) return a; return b;}</code>	<code># define MAX (a,b) ((a>b)? a: b)</code>
<code>inline float ABS (float a) { return (a>=0)? a: 0 - a; }</code>	<code># define ABS (a) ((a >= 0)? a: 0 - (a))</code>
<code>inline float PERCENT (float a, float b) {return 100.0 * a / b;}</code>	<code># define PERCENT (a,b) (100.0 * a / b)</code>
<code>inline void SWAP (int& a, int& b) {int t; t=a; a=b; b=t; return;}</code>	<code># define SWAP (a, b) {int t=a; a=b; b=t;}</code>
<code>inline bool ISODD (int x) {return (x %2 == 1)?1: 0; }</code>	<code># define ISODD (x) ((x %2 == 1)? 1: 0)</code>

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



函数嵌套

- 一个函数的函数体中包含一个或多个函数调用语句，即称为函数嵌套。
- 嵌套的含义是，如果函数 A 要调用函数 B，也就是说，函数 A 的定义要依赖于函数 B 的定义。因此函数 B 的定义或函数 B 的原型必须出现在函数 A 的定义语句之前。
- 函数 A 调用函数 B，在调用 A 的过程中，即执行 A 的函数体过程中调用 B，也就是中途把程序控制点转到 B 的函数体，在执行结束后再返回到 A 的函数体中。
- 函数嵌套调用所占用的空间（如赋值参数的创建等）用栈（stack）的方式管理。一般这种栈所分配的空间是有限的，因此函数互相嵌套的层数也是有限的。

函数嵌套

- 函数 main、f1、f2 的栈空间按照“后进先出”原则进行修改，下面给出在栈区它们的栈空间分配情况。

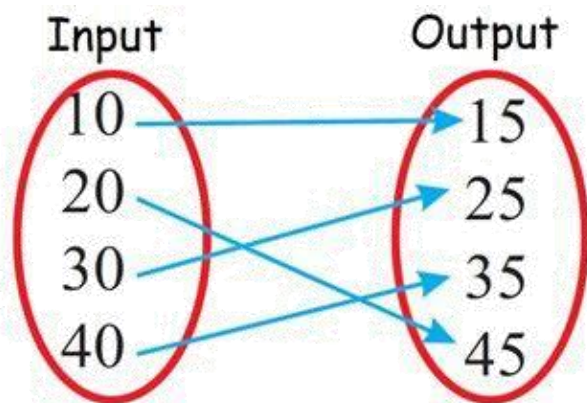
```
58 void f1 (int x, int y) {           64 void f2 (int p) {
59     int m = 2;                     65     if (p <= 100) cout << p;
60     x *= m ;                       66 }
61     y++;
62     f2 ((x+y)/m);
63 }
```

表 5.1 函数调用过程中的栈结构

函数中的变量和参数		栈 区
f2()	实参 p	38
		返回地址
		f1()运行状态
f1()	变量 m	2
	实参 x	23→46
	实参 y	30→31
		返回地址
		main()运行状态
main()	变量 a	23
	变量 b	7
	参数	无
		返回地址
		OS 运行状态

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



函数的递归

- 函数 A 在其函数体中直接包含对它自己的调用语句，这种调用称为直接递归调用，函数 A 称为（直接）递归函数。
- 函数 A 在其函数体中间接地包含对它自己的调用，例如 A 调用函数 B，但函数 B 又调用函数 A，则函数 A 称为（间接）递归函数。

```
float fac (int n){  
    if (n<2) return 1;  
    return fac (n-1)*n;  
}
```

- 递归函数的实际计算过程与递归次序相反，例如，设 $n=4$ ，计算 $\text{fac}(4)$ 的递归次序是： $\text{fac}(4) \rightarrow \text{fac}(3) \rightarrow \text{fac}(2) \rightarrow \text{fac}(1)$ ，而实际计算过程则正好相反： $\text{fac}(1)=1$ ， $\text{fac}(2)=\text{fac}(1)*2=2$ ， $\text{fac}(3)=\text{fac}(2)*3=6$ ， $\text{fac}(4)=\text{fac}(3)*4=24$ 。

函数的递归

- 反序输出：从键盘输入 10 个 int 型数，而后按输入的相反顺序输出它们。

```
68 void inv (int n)
69 {
70     int i;
71     cin>>i;
72     if (n>1)
73         inv(n-1);
74     else
75         cout<<"---- The result ----"<<endl;
76     cout<< i <<" ";
77 }
```

Input 10 integers:

1 2 3 4 5 6 7 8 9 10

---- The result ----

10 9 8 7 6 5 4 3 2 1

函数的递归

- 反序输出一个正整数的各位数值，如输入 231，应输出 132。

```
79 void conv (int n) {  
80     if (n<10) {  
81         cout << n;  
82         return;  
83     }  
84     cout << n%10;  
85     conv(n/10);  
86 }
```

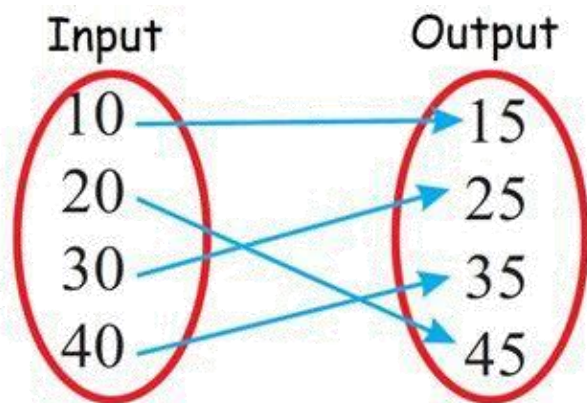
Input a positive number: 4781
1874

函数的递归

- 注意：
 - 无论是直接递归还是间接递归都必须保证在有限次调用之后能够结束。
 - 函数调用时系统要付出时间和空间代价，在环境条件相同的情形下，总是非递归程序效率较高。

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



函数重载

- 函数重载（overloading）实际上是函数名重载，即支持多个不同的函数采用同一名字。

```
int abs (int n) {return (n<0)? -n: n; }
```

```
float abs (float f ){ if (f<0) f= -f;   return  f; }
```

```
double abs (double d) {if (d<0) return -d; return  d;}
```

- 在程序中经常出现这样的情况：对若干种不同的数据类型求和，虽然数据本身差别很大（例如整数求和、向量求和、矩阵求和），具体的求和操作差别也很大，但完成不同求和操作的函数却可以取相同的名字（例如 sum、add 等）。
- 许多差别很大的打印函数可以都用同一个函数名 print，显示函数可以都用 display，从键盘或文件获取信息都称作 get，发送称为 send，接收称为 receive 等。
- 函数名的重载并不是为了节省标识符（标识符的数量是足够的），而是为了方便程序员的使用。

函数重载

- 实现函数的重载必须满足下列条件之一：
 - 参数表中对应的参数类型不同。
 - 参数表中参数个数不同。
 - 参数表中不同类型参数的次序不同。
- 注意：返回类型不同是不能区分重载函数的。

```
void print (int);
```

```
//整型
```

```
void print (point);
```

```
//类 point 的对象
```

```
int sum (int, int);
```

```
int sum (int, int, int);
```


函数重载

- 在定义同名函数时应注意：

- 返回类型不能区分函数

```
float add(int float);
```

```
int add(int float) ;           //错误
```

- 采用引用参数不能区分函数

```
void print (double);
```

```
void print (double &);         //错误
```

```
void print (const double &);   //错误
```

- 有些派生基本类型的参数虽然可以区分同名函数，但在使用中必须注意可能出现二义性。
- 包含可缺省参数时，可能造成二义性，应该尽量避免。

```
int abs(int);
```

```
unsigned int abs(unsigned int);
```

```
int sum (int a, int b, int c=0);
```

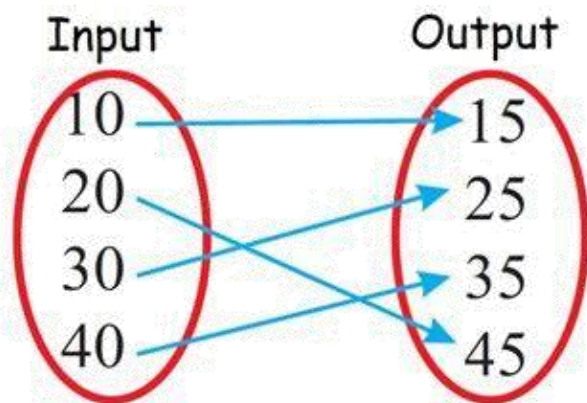
```
int sum (int a, int b);
```

函数重载

- 二义性的情形是指系统面对某一函数调用语句，有两个或两个以上同名函数与之匹配，应该避免；
- 遇到无准确匹配的函数定义时，C++系统并不马上按出错处理，它按下面的方式处理：
 - 通过数组名与指针变量、函数名与函数指针、某类型变量与**const** 常量之间的转换，去查看是否可实现匹配。
 - 把实参类型从低到高（按字长由短到长）进行基本类型及其派生类型的转换，再检查是否可匹配。
 - 查有无已定义的可变个数参数的函数，如有把它归为该函数。
 - 在进行上述尝试性的处理之后可能出现仍无匹配或匹配不唯一的情况，这时可能输出错信息或错误地运行。这种情况下，由于数据类型之间的复杂转换关系可能造成找不到与之匹配的函数定义。

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



可重载运算符

- C++语言中的运算符实际上是函数的方便表示形式，例如，算术运算符“+”也可以表示为函数形式：`int add (int a, int b) { return a+b;}`这时，`a+b` 和 `add (a,b)` 的含义是一样的。因此，逻辑上运算符也可以重载。

- C++语言规定，大多数运算符都可以重载，可重载的运算符如下：

(1) 单目运算符：

`-`、`~`、`!`、`++`、`--`、`new`、`delete`

(2) 双目运算符：

`+`、`-`、`*`、`/`、`%` (算术运算)

`&`、`|`、`^`、`<<`、`>>` (位运算)

`&&`、`||` (逻辑运算)

`==`、`!=`、`<`、`<=`、`>`、`>=` (关系运算)

`=` (赋值运算)

`+=`、`-=`、`*=`、`/=`、`%=` (赋值运算)

`^=`、`&=`、`|=`、`>>=`、`<<=` (赋值运算)

`,` (逗号运算)

`<<<`、`>>>` (I/O运算)

`()`、`[]` (其他)

可重载运算符

- 所列可重载运算符几乎包含了 C++ 的全部运算符集，例外的是：
 - 限定符 `.` 和 `::`。
 - 条件运算符 `?:`。
 - 取长度运算符 `sizeof`。

它们不可重载（不可赋予不同的操作）。

(1) 单目运算符：

`-`、`~`、`!`、`++`、`--`、`new`、`delete`

(2) 双目运算符：

`+`、`-`、`*`、`/`、`%`

(算术运算)

`&`、`|`、`^`、`<<`、`>>`

(位运算)

`&&`、`||`

(逻辑运算)

`==`、`!=`、`<`、`<=`、`>`、`>=`

(关系运算)

`=`

(赋值运算)

`+=`、`-=`、`*=`、`/=`、`%=`

(赋值运算)

`^=`、`&=`、`|=`、`>>=`、`<<=`

(赋值运算)

`,`

(逗号运算)

`<<`、`>>`

(I/O 运算)

`()`、`[]`

(其他)

可重载运算符

- 算术运算符、逻辑运算符、位运算符和关系运算符中的<、>以及<=、>=，这些运算都与基本数据类型有关，通过运算符重载函数的定义，使它们也用于某些用户定义的数据类型。
- 指针运算符&和*、下标运算符[]等运算所涉及的数据类型按C++程序规定，并非只限于基本数值类型。因此，这些运算符可以自动地扩展到任何用户定义的数据类型，不需作重载定义就可“自动”地实现重载。

可重载运算符

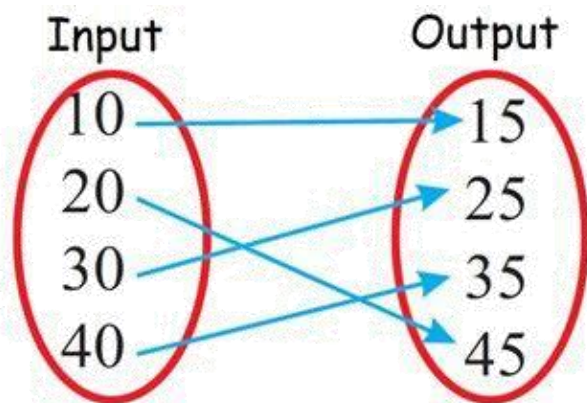
- 单目运算符++和--实际上各有两种用法：前缀增（减）量和后缀增（减）量。其运算符重载函数的定义当然是不同的，对两种不同的运算无法从重载函数的原型上予以区分：**函数名（operator ++）和参数表完全一样。为了区别前缀++和后缀++，C++语言规定，在后缀++的重载函数的原型参数表中增加一个int型的无名参数。**

前缀++:	<类型>operator ++()	//作为类成员
	<类型>operator ++(<类型>)	//作为类外函数
后缀++:	<类型>operator ++ (int)	//作为类成员
	<类型>operator ++(<类型>, int)	//作为类外函数

前缀++:	++a 或 a.operator++()
	operator++(a)
后缀++:	a++ 或 a.operator++(0)
	operator++(a, 0)

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



运算符重载

- 运算符的重载是一个特殊函数定义过程，这类函数总是以“operator<运算符>”作为函数名。
- 假设程序中定义了一个枚举类型的 Bool 类型：

```
enum Bool {FALSE, TRUE};
```

- 用运算符+(双目)、*(双目)、&(amp;单目)来表示或、与、非运算是十分方便的：

```
Bool operator + (Bool a , Bool b) {  
    if ( (a == FALSE) && (b == FALSE) ) return FALSE;  
    return TRUE;  
}  
  
Bool operator * (Bool a, Bool b) {  
    if ( (a==TRUE) && (b==TRUE) ) return TRUE;  
    return FALSE;  
}
```

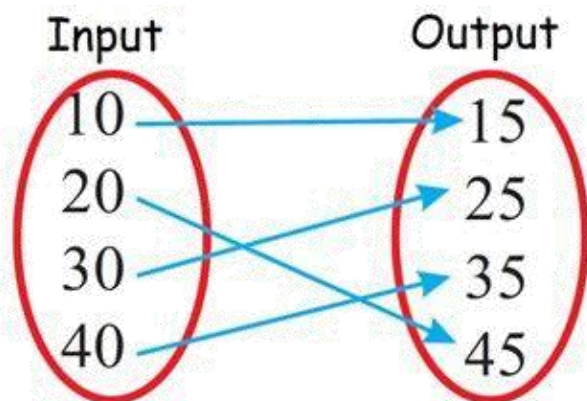
```
Bool operator & (Bool a) {  
    if (a==FALSE) return TRUE;  
    return FALSE;  
}  
  
b3 = b1+b2;  
b3 = b1*b2;  
b3 = &b1;  
b3 = (b1+b2) * FALSE;  
b3 = operator+ (b1,b2);
```

运算符重载

- 运算符重载函数的调用可有两种方式：
 - 与原运算符相同的调用方式，如上例中的 $b1+b2$ 、 $b1 * b2$ 等。
 - 一般函数调用方式，如 $b1+b2$ ，也可以写为 `operator+ (b1, b2)`。
被重载的运算符的调用方式，**优先级和运算顺序都与原运算符一致，其运算分量的个数也不可改变。**
- 运算符重载主要用于同类的形式定义的用户定义类型，例如，复数类型、集合类型、向量类型等，通过运算符重载把人们习惯的运算符引入到计算操作中，会收到很好的效果。

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



库函数的使用

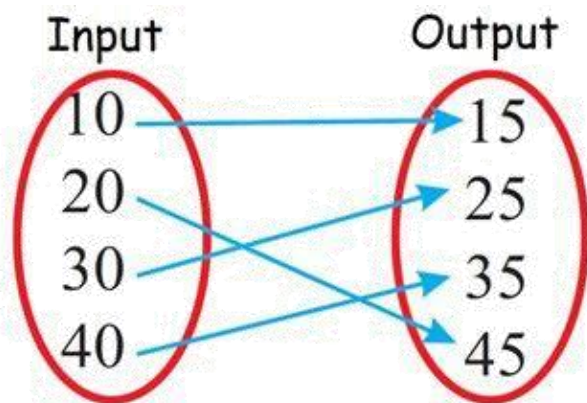
- 库函数又称标准函数，是 C++ 语言编译系统为用户提供的内部函数，其编写与一般用户定义的函数相同，程序员可在程序中直接使用，但是要在程序开头说明库函数所在的头文件名。

```
2  #include<iostream>
3  #include<math.h>
4  using namespace std;
```

- C++ 系统中有一个很大的标准函数库（和标准类库），包括许多在各种程序中常用的基本任务处理函数，这些库函数被分成不同的组，例如，数学计算、字符处理、字符串处理、I/O 操作、图形处理等，当 `#include` 命令包含了某个头文件时，实际上就是把这一组库函数的说明包含在程序中。

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归
- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - **SP 框架结构**
 - 函数间的数据传递
 - 变量与函数的作用域



SP 框架结构

- 按结构程序设计（SP）思想设计的程序结构称为 SP 框架。函数是 SP 框架的核心。
 - 一个 SP 框架的完整 C++ 程序由下面几部分组成：
 - 一个主函数：它可调用其他函数，但不能被调用。
 - 任意多个用户定义函数：都处于同一“等级”，可以互相调用。
 - 全局说明：在所有函数定义之外的变量说明和函数原型。
 - 预处理命令：在进行预处理后，这部分被取代。
 - 注释：只起方便阅读的作用，编译后被删除。

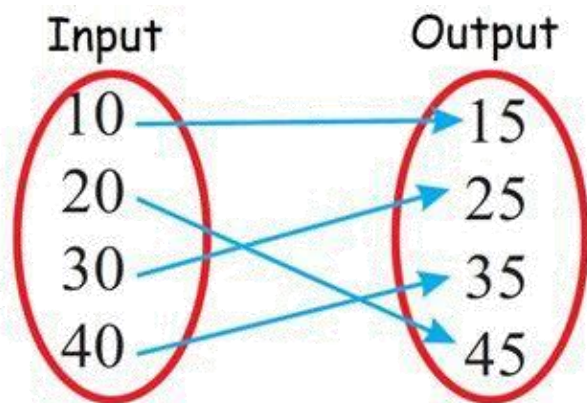
SP 框架结构

- 对于比较大的程序，可以把它们划分为几个程序文件，这些程序模块可能由一个或多个程序员编写，最简单有效的划分方法如下：
 - 根据主函数和各用户定义的函数的功能及相互关系，把它们划分成若干个**.CPP** 文件。
 - 按与每个**.CPP** 程序文件中的函数有关的全局说明组成一个或多个**.h**（头）文件。
 - 程序中使用库函数组成的若干**.CPP** 文件和对应的**.h** 文件。
- 在**预处理命令**的帮助下，一个 C++ 程序被划分为若干**.CPP** 和**.h** 程序文件。在包含命令的帮助下，这些文件形成了一个有机的整体。
- 包含若干全局说明的头文件，可能在各个模块中反复出现，这样一方面给程序员提供了方便，但是往往产生一个问题，同一个头文件在各程序模块中多次被包含的情形很容易出现。

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归

- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



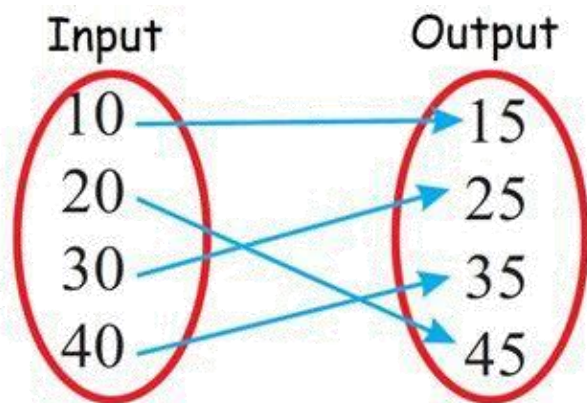
函数间的数据传递

- 函数是程序的基本模块，这种划分是十分必要的，一个程序中的各个函数又是互相联系的，这种联系体现为函数间的**数据传递**。
- C++语言为函数间的数据传递提供了多种机制，这些机制是学习程序设计所必须掌握的：
 - **通过赋值参数和返回语句**：从调用它的函数接收数据，通过返回（return）语句向调用它的函数传回数据，**两者都是单方向的**。
 - **通过全局变量**：全局变量的定义域可延续到整个程序执行结束，因此，**只要在函数中没有把该全局变量名说明为其他变量，在所有的函数中就可以直接访问它**，也就是说，函数间的数据传递还可以通过全局变量实现，这种**传递可以是双向的**。
 - **通过指针类型参数和引用参数**：通过指针类型的赋值参数给函数传进的不是数据本身，而是数据的地址，在函数体运行时，可以从该地址获得数据或输出数据，**其作用类似于全局变量，数据的传递可以是双向的**。
 - **通过函数的数组类型参数**：C++语言规定对于数组参数在调用时只传递该数组的首地址，实际上，就是该数组的指针，**其效果与指针类型参数相同，这种数据的传递也是双向的**。

目录

- 函数的说明与使用
 - 函数的说明
 - 函数的调用
 - 函数的返回
 - 函数的参数
 - 值调用与引用调用
 - 内联函数
- 函数的嵌套与递归
 - 函数嵌套
 - 函数的递归

- 函数与运算符的重载
 - 函数重载
 - 可重载运算符
 - 运算符重载
- 函数与 C++ 程序结构
 - 库函数的使用
 - SP 框架结构
 - 函数间的数据传递
 - 变量与函数的作用域



变量与函数的作用域

- 程序中出现的所有名字（标识符）都必须说明，每个名字（变量名、常量名、参数名、函数名、类名、对象名等）都在程序的一定范围内有意义，就是该名字的作用域。
- **外部存储属性与静态存储属性：**关键字 `extern`（外部存储属性）和 `static`（静态存储属性）可以规定所说明的变量名或函数名的作用域在一个程序文件范围内还是扩展到程序文件之外。

变量与函数的作用域

- **外部存储属性：**在所有的函数、类和名字空间之外说明的变量的作用域从被说明点开始，到所在的程序文件结束（注意，其生存期则是直到程序结束）。在另一个程序文件中如果需要使用同一个变量，则必须把这个变量说明为外部（**extern**）变量，表示这个变量的说明不在本程序文件中，而在其他文件中。

course3 >  extern_demo1.cpp

```
1  #include "extern_demo.h"
2
3  int t=2;
4  ✓ int main(){
5      t=5;
6      fa(t+1);
7      cout << t << endl;
8  }
9  ✓ void fa(int s){
10     cout << s + fb() << endl;
11 }
```

course3 >  extern_demo.h

```
1  #include<iostream>
2  using namespace std;
3
4  void fa(int s);
5  int fb();
```

course3 >  extern_demo2.cpp

```
1  #include "extern_demo.h"
2
3  extern int t;
4  int fb(){
5      t *= 2;
6      return t;
7  }
```

变量与函数的作用域

- C++语言规定，函数的说明和原型总是默认为外部（**extern**）的，换句话说，（类外）函数的作用域是整个程序，不需要做申明。

course3 >  extern_demo1.cpp

```
1  #include "extern_demo.h"
2
3  int t=2;
4  ∨ int main(){
5      |     t=5;
6      |     fa(t+1);
7      |     cout << t <<endl;
8      | }
9  ∨ void fa(int s){
10     |     cout << s + fb() <<endl;
11     | }
```

course3 >  extern_demo.h

```
1  #include<iostream>
2  using namespace std;
3
4  void fa(int s);
5  int fb();
```

course3 >  extern_demo2.cpp

```
1  #include "extern_demo.h"
2
3  extern int t;
4  int fb(){
5      |     t *= 2;
6      |     return t;
7      | }
```

变量与函数的作用域

- **静态存储属性：**对于变量和函数的说明，静态存储属性可以使其作用域为所在的程序文件，分以下 3 种情形：
 - **全局变量：**增加 `static` 说明，使其作用域仅限于本程序文件。
`int n;` 全局变量 `n` 的生存期为整个程序，作用域为本程序文件，但可以通过外部属性说明把它的作用域扩展到整个程序。
`static int m;` 静态全局变量 `m` 的生存期为整个程序，作用域为本程序文件，不可扩展。
 - **局部变量：**增加 `static` 说明，使其生存期扩展到整个程序运行过程：
`{ ... int n; ... }` 局部变量 `n` 的生存期和作用域限于其所在的语句块或函数体，所占用的空间位于临时数据区。
`{ ... static int m; ... }` 静态局部变量 `m` 的生存期为整个程序，程序第一次执行该语句时，为变量 `m` 分配空间，赋缺省初值 0，第二次以后执行该语句时，不再做初始化工作，每次执行时，变量 `m` 都指向同一地址。
 - **对于函数：**一般（类外）函数的生存期和作用域为整个程序，静态属性的函数的作用域被限于所在的程序文件，这时，该函数不能在其他程序文件中使用。

变量与函数的作用域

- C++程序中的名字（标识符）的作用域有下面几个等级：
 - **整个程序**：函数（或类）的作用域，全局变量（在 `extern` 说明帮助下）的作用域。
 - **程序文件**：在多文件结构的程序中，一个程序文件的范围小于整个程序，静态函数的作用域和静态变量的作用域都是其所在的程序文件。对于单文件程序，程序文件也就是整个程序。
 - **函数**：在函数内说明的局部变量的作用域在函数体内。
 - **程序块**：在任何一个复合语句、循环体（由符号“{”、“}”包含的若干程序语句）中说明的局部变量的作用域在该程序块内。

思考题

- 函数原型与函数定义有什么区别？什么情况下使用函数原型？
- 什么是形式参数（形参）？什么是实际参数（实参）？它们之间有什么联系？对应关系是什么？
- 赋值型参数与引用型参数有什么区别？函数调用的具体实施过程中，如何对它们进行不同的处理？
- 什么是内联函数？使用它的优点是什么？编译预处理的宏定义 `define` 是否可以实现类似于内联函数的功能？两者有什么区别？
- 什么是函数重载？使用它有什么好处？实现函数的重载必须满足什么条件？
- 什么是运算符的重载？各种不同的运算符是怎样进行重载的？运算符的重载与函数定义过程有什么联系？

思考题

- C++语言为函数间的数据传递提供了哪几种机制？从两种数据传递方向来看（从调用它的函数接收数据，“由上向下”；向调用它的函数传回数据，“由下向上”），每种机制的传递方式有什么不同？是单方向的还是双向的？
- 作用域和生存期的概念是否相同？是否有这样的变量，它在其作用域之外仍然保持占有其存储空间并保持其原有的值？
- C++程序中的名字（标识符）的作用域有哪几个等级？